

1. Database Setup

Run this first.

```
DROP DATABASE IF EXISTS joins_practice;
```

```
CREATE DATABASE joins_practice;
```

```
USE joins_practice;
```

2. Create Tables

customers table

```
CREATE TABLE customers (  
  customer_id INT PRIMARY KEY,  
  customer_name VARCHAR(50) NOT NULL,  
  city VARCHAR(50),  
  signup_date DATE  
);
```

products table

```
CREATE TABLE products (  
  product_id INT PRIMARY KEY,  
  product_name VARCHAR(100) NOT NULL,  
  category VARCHAR(50)  
);
```

orders table

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  customer_id INT,  
  product_id INT,
```

```
order_amount DECIMAL(10,2),
order_date DATE,
order_status VARCHAR(30)
);
```

payments table

```
CREATE TABLE payments (
  payment_id INT PRIMARY KEY,
  order_id INT,
  payment_mode VARCHAR(30),
  payment_status VARCHAR(30),
  paid_amount DECIMAL(10,2),
  payment_date DATE
);
```

3. Insert Sample Data

Insert customers

```
INSERT INTO customers
(customer_id, customer_name, city, signup_date)
VALUES
(1, 'Anurag', 'Bengaluru', '2026-01-10'),
(2, 'Stuti', 'Pune', '2026-02-15'),
(3, 'Rahul', 'Delhi', '2026-03-12'),
(4, 'Priya', 'Mumbai', '2026-04-05'),
(5, 'Neha', 'Bengaluru', '2026-05-20');
```

Insert products

```
INSERT INTO products
(product_id, product_name, category)
VALUES
(201, 'Laptop', 'Electronics'),
(202, 'Mouse', 'Accessories'),
```

```
(203, 'Keyboard', 'Accessories'),  
(204, 'Monitor', 'Electronics');
```

Insert orders

```
INSERT INTO orders  
(order_id, customer_id, product_id, order_amount, order_date, order_status)  
VALUES  
(101, 1, 201, 50000.00, '2026-06-01', 'Delivered'),  
(102, 1, 202, 1000.00, '2026-06-02', 'Delivered'),  
(103, 2, 203, 2500.00, '2026-06-03', 'Delivered'),  
(104, 99, 204, 15000.00, '2026-06-04', 'Delivered'),  
(105, 3, 999, 3500.00, '2026-06-05', 'Delivered'),  
(106, NULL, 201, 45000.00, '2026-06-06', 'Pending'),  
(107, 5, 202, 1200.00, '2026-06-07', 'Cancelled');
```

Insert payments

```
INSERT INTO payments  
(payment_id, order_id, payment_mode, payment_status, paid_amount, payment_date)  
VALUES  
(1001, 101, 'UPI', 'Success', 50000.00, '2026-06-01'),  
(1002, 102, 'Card', 'Success', 1000.00, '2026-06-02'),  
(1003, 103, 'UPI', 'Failed', 0.00, '2026-06-03'),  
(1004, 105, 'Wallet', 'Success', 3500.00, '2026-06-05'),  
(1005, 999, 'UPI', 'Success', 3000.00, '2026-06-08');
```

4. Check Data

```
SELECT * FROM customers;  
SELECT * FROM products;  
SELECT * FROM orders;  
SELECT * FROM payments;
```

Check counts:

```
SELECT COUNT(*) AS customer_count FROM customers;
SELECT COUNT(*) AS product_count FROM products;
SELECT COUNT(*) AS order_count FROM orders;
SELECT COUNT(*) AS payment_count FROM payments;
```

Expected counts:

```
customers = 5
products = 4
orders = 7
payments = 5
```

Practice Questions with Answers

Question 1

Show all valid orders with customer name and city.

Answer

```
SELECT
  o.order_id,
  c.customer_name,
  c.city,
  o.order_amount,
  o.order_date,
  o.order_status
FROM orders o
INNER JOIN customers c
  ON o.customer_id = c.customer_id;
```

Expected Logic

- Only orders with matching customer IDs will come.
- Order 104 will not come because customer ID 99 does not exist.
- Order 106 will not come because customer ID is NULL.

Question 2

Show customer name, product name, category and order amount.

Answer

```
SELECT
  c.customer_name,
  p.product_name,
  p.category,
  o.order_id,
  o.order_amount
FROM orders o
INNER JOIN customers c
  ON o.customer_id = c.customer_id
INNER JOIN products p
  ON o.product_id = p.product_id;
```

Expected Logic

- Order must have a valid customer.
- Order must have a valid product.
- Order 105 will not come because product ID 999 does not exist.

Question 3

Show customer name, order ID, payment mode and payment status.

Answer

```
SELECT
  c.customer_name,
  o.order_id,
  o.order_amount,
  pay.payment_mode,
  pay.payment_status
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id
```

```
INNER JOIN payments pay
  ON o.order_id = pay.order_id;
```

Expected Logic

- Only orders having matching customer and payment records will come.
 - Order 107 will not come because it has no payment record.
-

Question 4

Show only successful payments with customer details.

Answer

```
SELECT
  c.customer_name,
  c.city,
  o.order_id,
  o.order_amount,
  pay.payment_mode,
  pay.payment_status,
  pay.paid_amount
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id
INNER JOIN payments pay
  ON o.order_id = pay.order_id
WHERE pay.payment_status = 'Success';
```

Expected Logic

- Only successful payment records will come.
 - Failed payments will be removed by the **WHERE** condition.
-

Question 5

Find total amount spent by each customer.

Answer

```
SELECT
  c.customer_id,
  c.customer_name,
  SUM(o.order_amount) AS total_spent
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id
GROUP BY
  c.customer_id,
  c.customer_name;
```

Expected Logic

- Only customers who placed orders will come.
 - Priya will not come because she has no order.
-

Question 6

Find the number of valid orders placed by each customer.

Answer

```
SELECT
  c.customer_id,
  c.customer_name,
  COUNT(o.order_id) AS total_orders
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id
GROUP BY
  c.customer_id,
  c.customer_name;
```

Expected Logic

- Anurag will have 2 orders.
- Stuti, Rahul and Neha will have 1 order each.
- Priya will not come.

Question 7

Find total revenue generated from each city.

Answer

```
SELECT
    c.city,
    SUM(o.order_amount) AS total_revenue
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
GROUP BY c.city;
```

Expected Logic

- Revenue will be calculated only for matched customer orders.
- Orders with invalid or NULL customer IDs will not be counted.

Question 8

Show all Bengaluru customers who placed orders.

Answer

```
SELECT
    c.customer_name,
    c.city,
    o.order_id,
    o.order_amount
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
WHERE c.city = 'Bengaluru';
```

Expected Logic

- Only Bengaluru customers with matching orders will come.

- Anurag and Neha will appear.
-

Question 9

Find customers who used UPI payment mode.

Answer

```
SELECT DISTINCT
    c.customer_name,
    pay.payment_mode
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
INNER JOIN payments pay
    ON o.order_id = pay.order_id
WHERE pay.payment_mode = 'UPI';
```

Expected Logic

- **DISTINCT** is used because a customer may use UPI multiple times.
 - Only customers with UPI payments will come.
-

Question 10

Create a final report with customer, product, order and payment details.

Answer

```
SELECT
    c.customer_name,
    c.city,
    p.product_name,
    p.category,
    o.order_id,
    o.order_amount,
    o.order_status,
    pay.payment_mode,
    pay.payment_status,
```

```
    pay.paid_amount
FROM orders o
INNER JOIN customers c
    ON o.customer_id = c.customer_id
INNER JOIN products p
    ON o.product_id = p.product_id
INNER JOIN payments pay
    ON o.order_id = pay.order_id;
```

Expected Logic

Only records matching across all four tables will come.

Expected order IDs:

101
102
103

Question 11

Find total sales for each product category.

Answer

```
SELECT
    p.category,
    SUM(o.order_amount) AS total_sales
FROM orders o
INNER JOIN products p
    ON o.product_id = p.product_id
GROUP BY p.category;
```

Expected Logic

- Only orders with valid product IDs will be counted.
 - Order 105 will not be counted because product ID 999 is invalid.
-

Question 12

Find total successful paid amount by payment mode.

Answer

```
SELECT
    pay.payment_mode,
    SUM(pay.paid_amount) AS total_successful_amount
FROM payments pay
INNER JOIN orders o
    ON pay.order_id = o.order_id
WHERE pay.payment_status = 'Success'
GROUP BY pay.payment_mode;
```

Expected Logic

- Payment 1005 will not come because order ID 999 does not exist in orders.
 - Only successful payments are counted.
-

Question 13

Find customer-wise successful payment amount.

Answer

```
SELECT
    c.customer_name,
    SUM(pay.paid_amount) AS total_successful_paid_amount
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
INNER JOIN payments pay
    ON o.order_id = pay.order_id
WHERE pay.payment_status = 'Success'
GROUP BY c.customer_name;
```

Expected Logic

- Only successful payment records will be counted.

- Failed payments will not be counted.
-

Question 14

Find highest order amount placed by each customer.

Answer

```
SELECT
    c.customer_name,
    MAX(o.order_amount) AS highest_order_amount
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_name;
```

Expected Logic

- Customer-wise maximum order amount will be shown.
 - Customers without orders will not come.
-

Question 15

Find delivered orders with customer and product details.

Answer

```
SELECT
    c.customer_name,
    p.product_name,
    o.order_id,
    o.order_amount,
    o.order_status
FROM orders o
INNER JOIN customers c
    ON o.customer_id = c.customer_id
INNER JOIN products p
    ON o.product_id = p.product_id
WHERE o.order_status = 'Delivered';
```

Expected Logic

- Only delivered orders will come.
 - Orders must also have valid customer and product details.
-

Tricky Practice Questions

Question 16

Orders table has 7 records. After joining orders with customers, how many records will come?

Query

```
SELECT COUNT(*) AS join_count
FROM orders o
INNER JOIN customers c
  ON o.customer_id = c.customer_id;
```

Answer

5 records

Why?

- Order 104 has invalid customer ID 99.
 - Order 106 has NULL customer ID.
 - Both will be removed by INNER JOIN.
-

Question 17

After joining orders, customers, products and payments, how many records will come?

Query

```
SELECT COUNT(*) AS final_join_count
FROM orders o
```

```
INNER JOIN customers c
  ON o.customer_id = c.customer_id
INNER JOIN products p
  ON o.product_id = p.product_id
INNER JOIN payments pay
  ON o.order_id = pay.order_id;
```

Answer

3 records

Why?

Only orders 101, 102, and 103 match across all required tables.

Other orders are removed because of invalid customer, invalid product, NULL customer, or missing payment.

Question 18

Why is order ID 105 visible when joining orders with customers, but not visible when joining orders with customers and products?

Answer

Order 105 has a valid customer ID 3, so it appears when joining with customers.

But order 105 has product ID 999, which does not exist in products.

So it disappears when we also join products.

Question 19

Why is order ID 107 visible when joining orders with customers, but not visible when joining orders with payments?

Answer

Order 107 has a valid customer ID, so it appears when joining with customers.

But there is no payment record for order 107.

So it disappears when we also join payments.

Question 20

Does INNER JOIN match NULL values?

Query

```
SELECT
  o.order_id,
  o.customer_id,
  c.customer_name
FROM orders o
INNER JOIN customers c
  ON o.customer_id = c.customer_id;
```

Answer

No.

Order 106 has `customer_id = NULL`.

It will not match with any customer.

In SQL, `NULL = NULL` is not considered true.

Row Multiplication Practice

INNER JOIN does not always reduce rows.

It can increase rows also if the join key has multiple matches.

Extra DDL: customer_offers table

```
CREATE TABLE customer_offers (  
    offer_id INT PRIMARY KEY,  
    customer_id INT,  
    offer_name VARCHAR(100)  
);
```

Extra DML: Insert offers

```
INSERT INTO customer_offers  
(offer_id, customer_id, offer_name)  
VALUES  
(1, 1, 'Festive Discount'),  
(2, 1, 'Premium Customer Offer'),  
(3, 2, 'UPI Cashback');
```

Question 21

Join customers, orders and customer offers. Show customer name, order ID and offer name.

Answer

```
SELECT  
    c.customer_name,  
    o.order_id,  
    co.offer_name  
FROM customers c  
INNER JOIN orders o  
    ON c.customer_id = o.customer_id  
INNER JOIN customer_offers co  
    ON c.customer_id = co.customer_id;
```

Expected Logic

- Anurag has 2 orders.
- Anurag has 2 offers.
- So Anurag will produce:

2 orders x 2 offers = 4 rows

This is called row multiplication.

Question 22

Why are more rows coming after joining customer offers?

Answer

Because the join key is not unique.

One customer can have multiple orders.

The same customer can also have multiple offers.

When both tables are joined using customer ID, SQL creates all matching combinations.

Question 23

Find one row per customer with total order amount.

Answer

```
SELECT
  c.customer_id,
  c.customer_name,
  order_summary.total_amount
FROM customers c
INNER JOIN (
  SELECT
    customer_id,
    SUM(order_amount) AS total_amount
  FROM orders
  GROUP BY customer_id
) order_summary
ON c.customer_id = order_summary.customer_id;
```

Expected Logic

First aggregate orders at customer level.

Then join with customers.

This avoids unnecessary row multiplication.

Multiple Condition Join Practice

Extra DDL: city_targets table

```
CREATE TABLE city_targets (  
    city VARCHAR(50),  
    target_month DATE,  
    target_amount DECIMAL(10,2)  
);
```

Extra DML: Insert city targets

```
INSERT INTO city_targets  
(city, target_month, target_amount)  
VALUES  
( 'Bengaluru', '2026-06-01', 60000.00),  
( 'Pune', '2026-06-01', 10000.00),  
( 'Delhi', '2026-06-01', 8000.00),  
( 'Mumbai', '2026-06-01', 7000.00);
```

Question 24

Compare actual city-wise sales with city-wise monthly target.

Answer

```
SELECT  
    c.city,  
    SUM(o.order_amount) AS actual_sales,
```

```
t.target_amount
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id
INNER JOIN city_targets t
  ON c.city = t.city
  AND t.target_month = '2026-06-01'
GROUP BY
  c.city,
  t.target_amount;
```

Expected Logic

- Join customers with orders using customer ID.
 - Join city targets using city and month.
 - This is an example of joining with multiple conditions.
-

Interview-Based Practice Questions

Question 25

What is INNER JOIN?

Answer

INNER JOIN returns only matching records from both tables based on the join condition.

Question 26

Can INNER JOIN reduce records?

Answer

Yes.

INNER JOIN removes unmatched records, so output rows can be less than the original table.

Question 27

Can INNER JOIN increase records?

Answer

Yes.

If the join key has multiple matching records in one or both tables, output rows can increase.

Example:

2 rows in table A x 3 rows in table B = 6 rows

Question 28

Why are repeated rows coming after INNER JOIN?

Answer

Repeated rows can come because of one-to-many or many-to-many relationships.

Example:

One customer can place multiple orders.

So the customer appears once for every order.

Question 29

Should we use DISTINCT whenever repeated rows come after JOIN?

Answer

No.

First understand the business requirement and the grain of data.

If repeated rows are valid because of one-to-many relation, DISTINCT may hide the real logic.

Question 30

When should we not use INNER JOIN?

Answer

Do not use INNER JOIN when unmatched records are also required.

Examples:

- All customers including customers with no orders
- Orders with invalid customer IDs
- Products that were never sold
- Payments without valid orders

For these cases, use LEFT JOIN, FULL OUTER JOIN, or ANTI JOIN based on the requirement.

Final Revision

- INNER JOIN returns only matching records.
- It removes unmatched records.
- It can reduce row count.
- It can also increase row count if keys are repeated.
- NULL join keys do not match.
- Repeated rows are not always duplicates.
- Always understand the grain before using DISTINCT.
- Use INNER JOIN only when matched records are required.